Quantizing Small-Scale State-Space Models for Edge AI

Leo Zhao *
Institute of Neuroinformatics
University of Zürich and ETH Zürich
lezhao@ethz.ch
Zürich, CH

Tristan Torchet *
Institute of Neuroinformatics
University of Zürich and ETH Zürich
tristan.torchet@uzh.ch
Zürich, CH

Melika Payvand
Institute of Neuroinformatics
University of Zürich and ETH Zürich
melika@ini.uzh.ch
Zürich, CH

Laura Kriener §
Institute of Neuroinformatics
University of Zürich and ETH Zürich
laura.kriener@uzh.ch
Zürich, CH

Filippo Moro §
Institute of Neuroinformatics
University of Zürich and ETH Zürich
filippo.moro@uzh.ch
Zürich, CH

***Abstract*—State-space models (SSMs) have recently gained attention in deep learning for their ability to efficiently model long-range dependencies, making them promising candidates for edge-AI applications. In this paper, we analyze the effects of quantization on small-scale SSMs with a focus on reducing memory and computational costs while maintaining task performance. Using the S4D architecture, we first investigate post-training quantization (PTQ) and show that the state matrix $\mathcal{A}$ and internal state x are particularly sensitive to quantization. Furthermore, we analyze the impact of different quantization techniques applied to the parameters and activations in S4D architecture. To address the observed performance drop after Post-training Quantization (PTQ), we apply Quantization-aware Training (QAT), significantly improving performance from 40% (PTQ) to 96% on the sequential MNIST benchmark at 8-bit precision. We further demonstrate the potential of QAT in enabling sub 8-bit precisions and evaluate different parameterization schemes for QAT stability. Additionally, we propose a heterogeneous quantization strategy that assigns different precision levels to model components, reducing the overall memory footprint by a factor of 6x without sacrificing performance. Our results provide actionable insights for deploying quantized SSMs in resource-constrained environments.**

***Index Terms*—State-space Models, Quantization, Edge AI**

I. Introduction

State-space models (SSMs) [1]–[3] have recently garnered significant attention as highly effective sequence-modeling architectures. SSMs offer a principled framework for capturing long-range dependencies through structured linear dynamical systems, acting as promising alternatives for Recurrent Neural Networks (RNNs), which struggle to achieve the same performance, and attention-based models, which scale unfavorably in computational complexity. Recent advances, such as the Structured State-space Sequence model (S4) [2] and its more efficient variant S4D [4], have demonstrated strong empirical results on a variety of sequence tasks while maintaining favorable computational properties, including linear time and space complexity during training. For inference, these models can be run in the recurrent form with linear time and constant space complexity. These advantages render SSMs particularly attractive for edge-AI applications with strict constraints on latency, memory, and power consumption. By combining modeling capacity with efficient inference, SSMs emerge as a promising class of models for real-time, on-device processing of temporal data such as speech [5]–[7], bio-medical signals [8], [9], and sensor streams.

Despite their parameter efficiency, deploying SSMs on edge devices remains a challenge due to limited available hardware resources in such environments [10] (Fig. 1, left). Methods to reduce the computational complexity of SSMs include sparsifying intra-layer communication by introducing binary (*Heaviside*) activation functions [11], [12], which can be viewed as a *thresholding* operation or *spiking* non-linearity in a Spiking Neural Network. We focus instead on quantization, which has emerged as a key technique to address these constraints by reducing the bit-width of model weights and activations [13], [14]. Quantization not only reduces memory usage but also enables low-precision arithmetic, which simplifies computations and reduces energy consumption [15]. However, the effectiveness of quantization varies widely across model architectures and applications. In models like SSMs, which maintain and update internal states over time, aggressive quantization can impair stability and degrade overall performance [16]. Previous works investigated the quantization of large-scale SSMs for efficient deployment on Graphical Processing Units (GPUs) [17]–[19], using only PTQ techniques. Another work analyzed the performance of a quantized S5 SSM model [16], while [20] implemented an SSM network on the Loihi neuromorphic chip [21].

To the best of our knowledge, here for the first time we comprehensively study the effects of both PTQ and QAT on small-scale, low-precision SSMs. Understanding how different parts of the model tolerate quantization and how to design

* equal contribution as first author; § equal contribution as last author

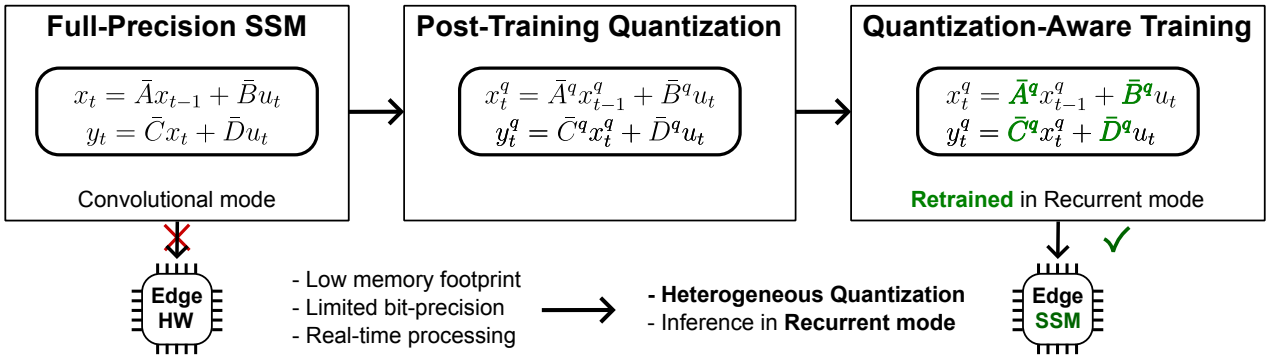


Fig. 1: Quantization of small-scale SSM networks for edge Artificial Intelligence (AI). Edge AI processors impose constraints on the SSM models, such as limited memory footprint and limited bit-precision arithmetic. We adapt the pre-trained convolutional-mode SSM to these constraints first with PTQ, showing that this incurs a consistent performance drop. We then recover the lost performance with recurrent-mode QAT and a heterogeneous quantization of the SSM parameters that minimizes the memory footprint while maximizing performance.

robust quantization schemes is essential for enabling reliable and efficient edge deployment. Furthermore, existing works do not address the effects of quantization in recurrent mode, which is essential for deployment in resource-constrained hardware.

In this work, we present a systematic study of quantization using fundamental techniques in a small-scale recurrent S4D trained on the sequential MNIST (sMNIST) task, a standard benchmark for sequence modeling.

A. State-space Models for Edge AI

Several variations of SSMs have been developed lately [1]–[3], offering different compromises between computational efficiency and performance. These models are built around the state-space model originating in control theory that describes continuous-time linear time-invariant (LTI) systems, which can be expressed by the following equations:

$$\begin{aligned} \dot{x}(t) &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) + Du(t) \end{aligned} \tag{1}$$

where $x(t) \in \mathbb{C}^N$ is the latent state, $u \in \mathbb{R}$ is the input and $y \in \mathbb{R}$ is the output. $A \in \mathbb{C}^{N\times N}, B \in \mathbb{R}^{N\times 1}, C \in \mathbb{R}^{1\times N}, D \in \mathbb{R}^{1\times 1}$ are the *state-space matrices*. N is the state size. We call A, B, C, D the *state transition matrix*, the *input matrix*, the *output matrix* and the *feedthrough matrix*, respectively.

In practice, u and y are discrete input and output sequences, and the time-continuous form of the SSM must be discretized during inference, resulting in the equations seen in 1. The discrete forms of the state-space matrices are denoted with a bar, e.g. $\bar{A}$. Typical discretization methods are the zero-order hold (ZOH) or bilinear method.

In the S4 architecture, the continuous-time state-space matrices are trained along with the time step Δ, then discretized during the forward pass. The state-space matrices in S4D are initialized according to the HiPPO framework [1], which compresses past input history into a fixed-size hidden state using orthogonal projections designed to optimally preserve recent information [22]. In particular, the $\bar{A}$ matrix in S4D is simplified to the diagonal form for computational efficiency. The SSM layer consists of H parallel and independent SSMs, where H is the number of heads. The outputs of each SSM are passed through a non-linearity and mixed to produce the layer output. The depth of the network consists of D stacked SSMs, where D is the depth.

Algorithm 1 S4D Architecture with D Layers and H Heads

Input: Sequence $u^0 = \{u_1, u_2, \ldots, u_L\}$
for $l = 1$ to D **do** % Repeat SSM layers
  for $h = 1$ to H **do** % Multi-head parallelism
    Compute $y_l^h = \text{SSMhead}(u^l)$
  end for
  Concatenate heads: $y^l = \text{Concat}(y_l^1, \ldots, y_l^H)$
  Apply non-linearity: $y^l = \text{GeLU}(y^l)$
  Update input: $u^{l+1} = \text{GLU}_l(\text{Conv1D}_l(y^l))$
end for
Output: $out = \text{Linear}(u^D)$

A summary of the S4D architecture is provided in Algorithm 1.

TABLE I: Full precision baseline results on sMNIST.

Model	#Params	Test accuracy[%]	StdDev[%]
S4D-16h	3,850	97.200	0.250
S4D-64h	21,514	98.792	0.119
S4D-256h	184,330	99.248	0.066
S4D-512h	630,794	99.270	0.061

II. Method

The S4D offers two modes of execution: the *convolutional* and *recurrent* modes. The first exploits parallel computation in GPUs, but requires processing the entire input sequence in parallel, a requirement that likely is not satisfied in an edge case where inputs are directly streamed from a sensory system. Instead, the recurrent mode treats each input data

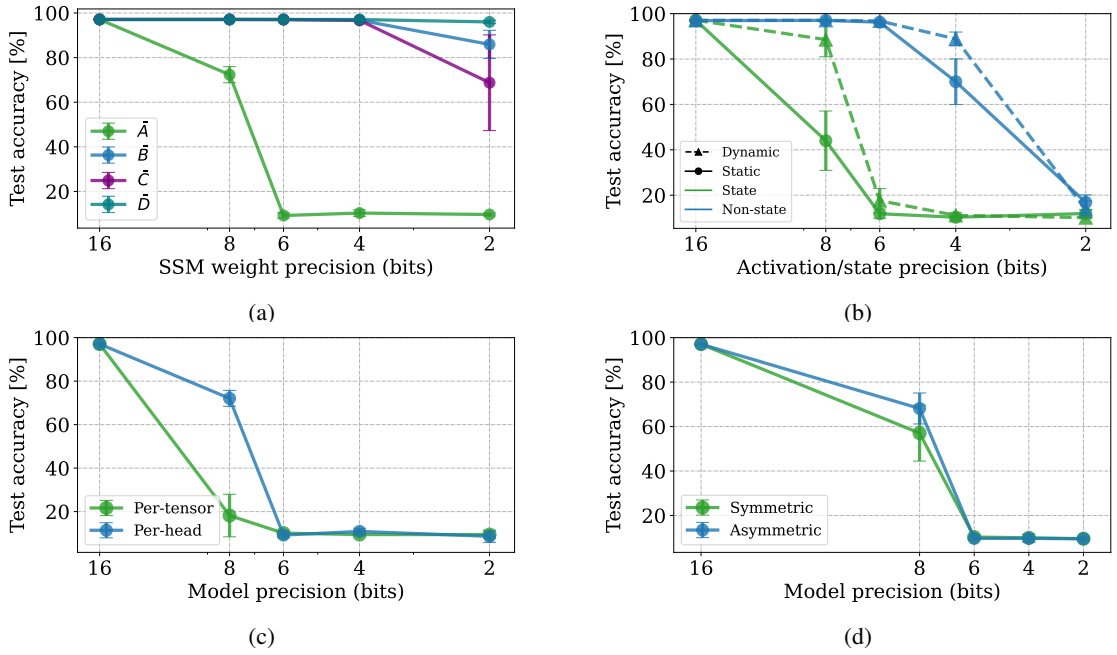


Fig. 2: Accuracy on sMNIST using PTQ of different S4D model components with 16 heads and different quantization modes. (a) Quantization of a single state-space matrix ($\bar{A}$, $\bar{B}$, $\bar{C}$, or $\bar{D}$). (b) Quantization of state or all other activations (non-state) for *static* and *dynamic* calibration schemes. (c) Effect of quantization granularity at given model bit precision. (d) Effect of *symmetric* and *asymmetric* quantization at given model bit precision. All experiments are performed on 5 different seeds, showing the standard deviations as error bars.

point sequentially, and is thus compatible with edge processing for inference. The propagation of quantization errors or noise in the recurrence during execution at the edge is also more accurately modeled by the recurrent form of the model. For comparison, [16] quantizes the S5 model run with the *associative scan* technique. This technique computes the SSM output in parallel and ignores the recurrently propagated error of the quantized state x computed in recurrent mode. We are thus interested in investigating the effect of quantization on the model in recurrent mode.

We first pre-train our convolutional S4D models on the sequential MNIST task (sMNIST) in full precision and obtain the baseline classification accuracies summarized in Table I. For all model sizes, we train for 30 epochs with a learning rate of $1\mathrm{e}{-3}$, following the training procedure described in the S4D paper [4]. In convolutional mode, the SSM output y is computed through the convolution $y = \bar{K} * u$, where the convolutional kernel $\bar{K} = \sum_i^n C\bar{A}^i\bar{B}$ is efficiently computed exploiting the diagonality of $\bar{A}$. We then switch the model to its recurrent form and proceed with the quantization of the S4D parameters for PTQ.

For the SSM, we directly quantize the discrete state-space matrices. As $\bar{A}$, $\bar{B}$, and $\bar{C}$ are complex-valued tensors, we separate the real and imaginary parts but always quantize both parts according to the same scheme to reduce the complexity of cross-multiplications in future hardware implementations. Moreover, quantizing the state x, which is calculated recurrently, is necessary in recurrent mode to take advantage of limited bit-precision in the SSM parameters. Our experiments show that the performance of the quantized S4D models is extremely sensitive to the precision and stability of the state x. As such, we heuristically clip the real and imaginary components of the state to the interval $[-50, +50]$ to prevent divergence over time. Activations outside the SSM are real-valued. The SSM layer also includes a GeLU non-linearity, which we quantize as $\mathrm{qGELU}_q(x) = x \cdot \min(\max(0, x+2), 1)$, following the method developed in [16].

Quantization can be employed with an enormous selection of techniques. We thus select the most fundamental, commonly used quantization parameters for our experiments. We map the full-precision range to the quantized range uniformly, using a 99.999% percentile-based range calibration to remove outliers. The quantized range is always defined by the range representable by a signed integer at a given bit-precision.

We vary quantization *granularity*, *symmetry*, and *calibration mode*: *Granularity* specifies whether the quantization operation is to be performed on the entire tensor (*per-tensor*) or individually on different channels of the tensor (*per-channel*). In the case of the S4D, H independent SSMs are stacked in parallel, as observed in Algorithm 1. Formally, the weights, states and activations of each head can be aggregated into respective tensors with an H dimension. Per-channel or per-tensor quantization can then be applied to this tensor. In this case of the SSM, we emphasize that per-channel quantization of the state-space matrices is performed over the H dimension by referring to it as *per-head*. *Symmetry* describes the assump-

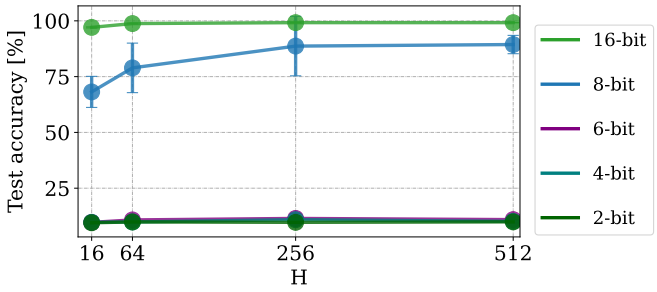


Fig. 3: Effect of scaling the model width on PTQ performance. H is the number of heads in the SSM layer.

tions on the full-precision range. The full-precision range can be taken to be symmetric around zero (*symmetric*), which is then mapped to the quantized range through a linear scaling, or as an arbitrary range that requires an affine mapping to the quantized range (*asymmetric*). The latter is more accurate, but also more computationally and memory-expensive as it complicates quantized arithmetic and requires storing the zero-point of the mapping. Finally, the *calibration mode* determines whether activations and other runtime-computed values (such as state) are quantized *statically*, by pre-calibrating to obtain range statistics fixed during deployment, or *dynamically* by adjusting the statistics each time new inputs are presented. The latter mode is impractical for deployment in edge processing cases, and we generally implement static quantization unless specified otherwise.

For QAT, we perform 10 additional epochs *in recurrent mode* on the pre-trained quantized model with a learning rate of $1\mathrm{e}{-4}$ (this is also known as quantization-aware fine tuning (QAFT)). As is customary, we utilize the straight-through gradient estimator (STE) [23] to allow backpropagation of errors through non-differentiable operations. To prevent exploding gradients in the recurrence, we further constrain the gradients to the range $[-1000, 1000]$. All experiments use asymmetric, per-head, and static quantization schemes.

A crucial element for consideration during QAT is the parameterization of the state-space matrices, which is important for model stability during training. Parametrization refers to the functional form utilized to express a parameter; in the case of SSMs, it is used to condition the training of $\bar{A}$ and avoid instability in the state. In the original S4D model, the state-space matrices (and Δ) are trained in continuous-time, where A is further parameterized by independently training the negative log of the real component and the imaginary component. In our case, one can directly train the quantized discrete state-space matrices or continue to train the continuous-time state-space matrices in the original parametrization. We refer to these parameterizations as *Discrete* and *Continuous* (or *Original*), respectively. Additionally, we experiment with *Frozen* $\bar{A}$, meaning that we train the discrete state-space matrices, but apply no updates to $\bar{A}$ during QAT.

To summarize, we always apply the quantization function $Q(\cdot)$ after the discretization function $\mathrm{Disc}(\cdot)$. In our

TABLE II: State transition matrix parameterization schemes for QAT. "Discrete" trains $\bar{A}$ directly, "Continuous" uses the original S4D parameterization and trains the continuous-time A.

Name	Trainable Parameters	Parameterization of $\bar{A}$
Discrete	$\bar{A}, \bar{B}, \bar{C}, \bar{D}$	$\bar{A} = \bar{A}$
Continuous (original)	A_{Re}, A_{Im}, B, C, D, Δ	$A = -e^{A_{Re}} + iA_{Im}$ $\bar{A} = \mathrm{Disc}(\bar{A}, \Delta)$

experiments, we use ZOH discretization. For the continuous parametrization of $\bar{A}$, the quantized and discretized $\bar{A}_q$ is given as:

$$\bar{A}_q = Q(\mathrm{Disc}(\bar{A})) = Q(e^{A\Delta}) \tag{2}$$

For all experiments, we apply *fake-quantization* [15] to the S4D parameters, which simulates the effects of quantization through rounding and clipping. The quantization function $Q(\cdot)$ rounds and clips its argument to the full-precision values representable by a given bit-precision.

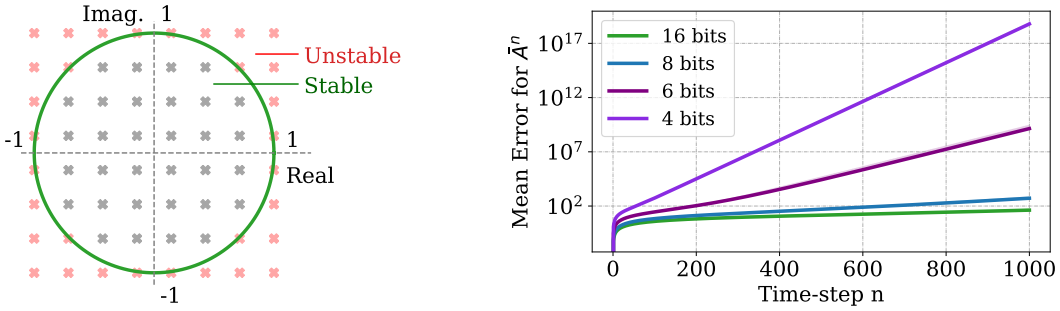
All PTQ and QAT experiments are initialized with 5 distinct seeds. When applicable, standard deviations of the results of different seeds are shown as error bars.

III. Results on Post-Training Quantization

In this section, we analyze the effect of quantizing the pre-trained full-precision S4D models with a thorough ablation study to identify the sensitivity of different model components to quantization parameters and bit-precision. We perform all the experiments in this section on the smallest S4D model with 16 heads (S4D-16h), as its small size exacerbates parameter sensitivity to quantization. Unless specified otherwise, we quantize weights and activations with a static, per-head, and asymmetric quantization scheme.

a) SSM weights: We first analyze the effect on model performance of independently quantizing each state-space matrix ($\bar{A}$, $\bar{B}$, $\bar{C}$, and $\bar{D}$). All other parameters and activations remain in full-precision. As shown in Fig. 2a, the state transition matrix $\bar{A}$ is the most critical in determining the performance of the SSM, and quantizing it to 8-bit precision already incurs a heavy performance penalty. Quantizing $\bar{B}$, $\bar{C}$, and $\bar{D}$ has little effect on performance down to 4-bit precision. In general, $\bar{C}$ is slightly more sensitive than $\bar{B}$ and $\bar{D}$.

b) State and activations: Next, we focus on the runtime-computed values, including all activations, in the S4D architecture, with emphasis on the SSM state. For this analysis, we evaluate both *static* and *dynamic* quantization modes for the either the state with all other runtime-computed values in full precision, or all runtime-computed values with only the state in full precision. Recall that the *static* mode is easier to implement in hardware, while the *dynamic* mode is expected to be more precise. Similar to the $\bar{A}$ parameter, the state x precision is a crucially important factor in post-quantization performance (Fig. 2b). In fact, 8-bit quantization suffices to disrupt performance. Moreover, *dynamic* quantization mode



(a) (b)

Fig. 4: Quantization of complex-valued parameters. (a) Real and Imaginary parts are quantized independently, here with 3-bit precision, with quantized values represented with crosses. The red area around the unit circuit is unstable, while crosses landing within the circle are stable. (b) Mean Euclidean distance (mean error $||A^n_{fp} - A^n_q||_2$) between the full-precision A_{fp} and the quantized A_q for different quantization bit precisions, and different time steps (powers of n).

performs much better than the *static* mode at the boundary precision before model failure, but it is impractical in resource-constrained hardware at the edge. The other runtime-computed values in Fig. 2b can be more aggressively quantized, and 6 bits are enough to maintain high accuracy.

c) Quantization granularity: We further explore quantization granularity. We analyze the effect of the two quantization modes in Fig. 2c, where all parameters are at quantized at the same bit-precision. In the *per-tensor* case, classification accuracy drops to chance level already at 8-bit precision. As expected, *per-head* quantization yields better performance than *per-tensor*, as it often yields higher-resolution quantization ranges customized for each SSM head. This is mainly visible at the 8-bit boundary of model failure.

d) Quantization symmetry: We apply either *symmetric* or *asymmetric* quantization to all parameters and activation quantized at the same bit precision. As expected, the model's performance significantly drops at 8-bit precision. *Asymmetric* quantization yields slightly better classification accuracy than the *symmetric* case, but the difference is only noticeable at 8-bit precision.

e) Scaling up to wider models: Our previous experiments are performed with the smallest S4D size, where the effects of quantization are most easily observable. We observed that the state transition matrix $\bar{A}$ and state x, under PTQ, require a minimum precision of 8 bits. Here, we investigate whether this boundary holds when scaling up the width of the SSM layer by increasing the number of heads. We increase the number of heads from 16 to 512, thus increasing the model size from about 4k to 630k, while quantizing all weights and activations at a precision varying from 2 to 16 bits. Results are shown in Fig. 3. Even for larger models, bit-precisions lower than 8-bit completely hinders the model performance. Only at the 8-bit critical point, before catastrophic model failure, does the model size positively correlate with classification accuracy. Nevertheless, scaling model width does not permit the 8-bit model to approach the performance achieved by the 16-bit model.

In summary, larger models generally do not permit more aggressive PTQ-quantization, except specifically at the critical bit-precision. When using only PTQ, the catastrophic effects of poor quantization schemes persist for larger models. Classification accuracy remains limited by the requirement of high bit-precision in the SSM layer.

IV. Stability of the State Transition Matrix State

The state transition matrix $\bar{A}$ and state x are extremely sensitive to quantization. In this section, we analyze the cause of this sensitivity. Recall that $\bar{A}$ is expressed in the complex domain, and we separate its real and imaginary components before quantization. However, we always quantize both components with the same scheme, mapping the full-precision range to the quantized range with the same scaling and offset. This is depicted in Fig. 4a where the quantized space (a uniform Cartesian grid) is marked as crosses in the complex plane.

The LTI system (Eq. (1)) can only be stable when the real components of all eigenvalues of A are strictly negative. In the discrete system, the eigenvalues of $\bar{A}$ must have magnitudes strictly less than 1. In the case of the S4D model, all elements of the $\bar{A}$, which is already diagonal, must land within the unit circle (in green in Fig. 4a). However, even if all full-precision values are within the unit circle, the quantized space nevertheless allows values outside the unit circle (red crosses in Fig. 4a), resulting in potential instability. With low bit-resolution, the number of stable quantized values close to the unit circle reduces, and the entries in $\bar{A}$ are quantized to the nearest representable value. If the nearest available quantized value for an entry of $\bar{A}$ falls outside the unit circle, then the SSM will be unstable. Errors in the quantization of $\bar{A}$ are particularly critical, as the state x is repeatedly multiplied by $\bar{A}$ within the recursion. Therefore, errors due to the quantization of $\bar{A}$ are amplified by the power of n, where n is the time step, causing x to explode. This is shown in Fig. 4b, which approximates quantization error by showing the mean absolute error between the quantized and full-precision $\bar{A}^n$.

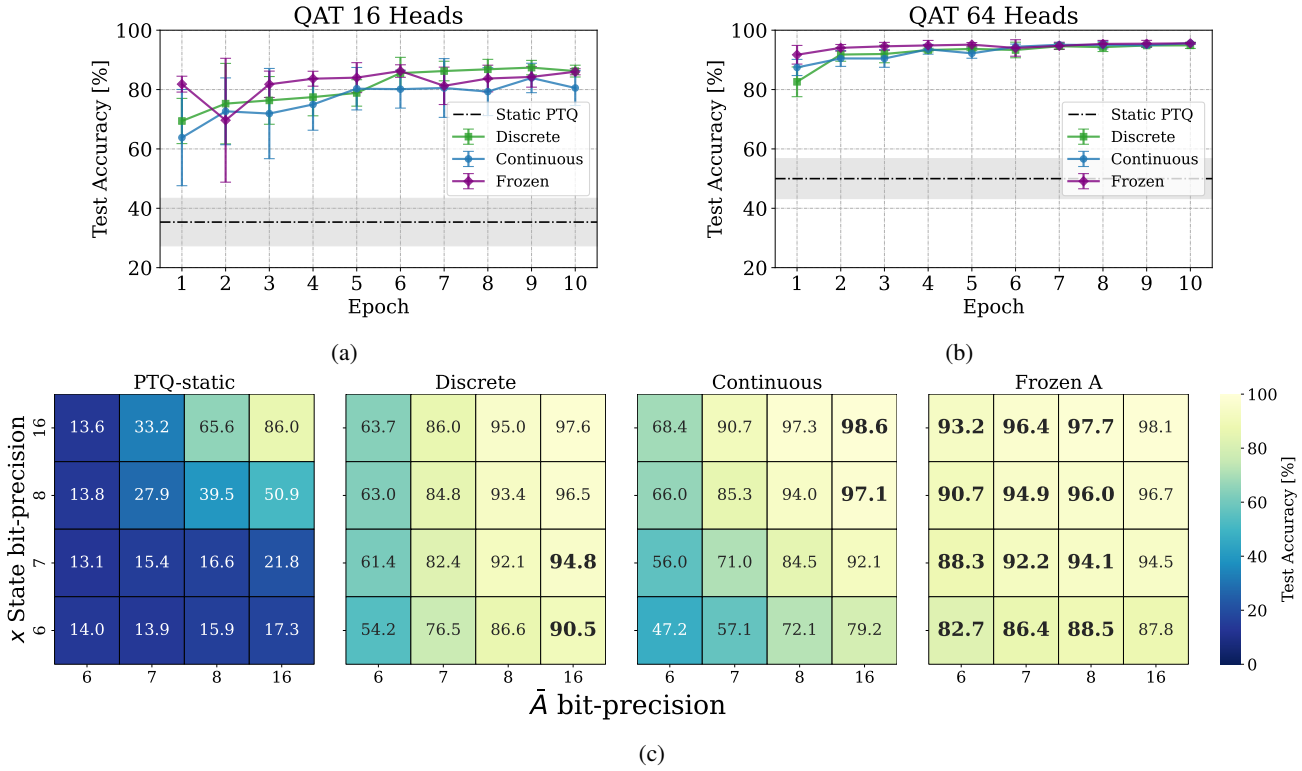


Fig. 5: QAT on S4D model with 16 (a) and 64 (b) heads, with all parameters and activations quantized at 8 bits, using asymmetric and static schemes. Experiments in (a) and (b) are performed on 5 different seeds, and standard deviations are shown with error bars. (c) Performance of QAT for S4D with 64 heads for different bit-precision of $\bar{A}$ and x.

An additional advantage of operating the model in recurrent mode is that the state can be constrained after each time step. Rather than explicitly constraining the eigenvalues of $\bar{A}$ within the unit circle, we found that simply clipping x within a bounded range at each time step offers a simple, robust, and general solution to the instability problem during both PTQ and QAT.

V. Results on Quantization-Aware Training

We perform QAT to improve the performance of the quantized S4D models. Starting from the pre-trained quantized model, we train both the small (16 heads) and medium (64 heads) quantized models in recurrent mode. We analyze three distinct parameterizations of $\bar{A}$: "Discrete", "Continuous", and "Frozen $\bar{A}$", discussed in Section II. In all cases, the classification accuracy greatly increases after just one QAT epoch (Fig. 5(a), (b)), and it reaches convergence within 10 epochs. On the smaller model size (16 heads), the "Continuous" S4D parametrization underperforms compared to the "Frozen $\bar{A}$" and "Discrete" ones. On the 64-head model, "Frozen $\bar{A}$" parameterization converges faster and reaches slightly higher accuracy.

a) Heterogeneous quantization scheme: Our PTQ analysis showed that different weights and activations in the S4D model display markedly different sensitivities to quantization. Consequently, we introduce a heterogeneous scheme that quantizes parameters and states at different bit-precisions. Such a scheme is necessary to optimize memory usage for hardware deployment.

Outside the SSM layer, we quantize all parameters to 4 bits. Within the SSM layer, we also apply 4-bit quantization to the $\bar{B}$, $\bar{C}$, and $\bar{D}$ state-space matrices. All activations, except the state, are quantized to 6 bits. Meanwhile, we investigate how far one can reduce the bit-precision for $\bar{A}$ and x.

Figure 5c shows final test accuracies after QAT for the three proposed parameterizations of $\bar{A}$ and compares them with the accuracies after PTQ. At high-precision $\bar{A}$ and x, continuous parameterization performs best, while at high-precision $\bar{A}$ and low-precision x, directly training the state space matrices with discrete parameterization yields the best performance. Most importantly, the frozen $\bar{A}$ parameterization consistently outperforms the other two parameterizations on lower $\bar{A}$ precisions, which is the most hardware-relevant domain.

While this is surprising, as the model trains fewer parameters, this parameterization could be a pragmatic choice in cases where low bit-precision quantization does not allow fine-tuning of the state transition matrix $\bar{A}$. The performance of this parameterization during QAT suggests that the best choice of quantized $\bar{A}$ at low precisions is the one that is closest to its floating-point value.

To summarize, in Table III we report the results obtained during the QAT analysis, including the model's classifica-

tion accuracy and the memory savings compared to the full-precision baselines. Note that the 64-head model with heterogeneous quantization and 8-bit precision in the state transition matrix $\bar{A}$ and state x initially perform at around 40% after PTQ, but after QAT attains 95.99% classification accuracy. This performance is achieved while reducing the full-precision model's memory footprint by 84.05% , equal to a 6x compression.

TABLE III: Summary of memory footprint savings from QAT results. Accuracy and standard deviation of the best-performing parameterization and quantization scheme on sMNIST. "W[X]A[Y]" denotes bit-precision of X for weights, Y for activations. $\bar{A}$ and x, if quantized differently from other weights and activations, are also denoted explicitly with their precisions.

Model	Quantization	Accuracy [%]	Mem Savings [%]
S4D-16	FP	97.20 ± 0.25	-
	PTQ W4A6 $\bar{A}8x8$	29.36 ± 14.58	82.96
	W8A8	92.64 ± 0.62	76.55
	W4A6, $\bar{A}16x16$	96.59 ± 0.30	73.54
	W4A6, $\bar{A}8x8$	88.39 ± 1.40	82.96
	W4A6, $\bar{A}6x6$	57.04 ± 3.13	85.31
S4D-64	FP	98.79 ± 0.12	-
	PTQ W4A6 $\bar{A}8x8$	39.46 ± 14.98	**84.05**
	W8A8	97.66 ± 0.30	76.29
	W4A6, $\bar{A}16x16$	98.62 ± 0.08	76.70
	W4A6, $\bar{A}8x8$	**95.99** ± 0.56	**84.05**
	W4A6, $\bar{A}6x6$	84.21 ± 2.53	85.89

VI. Conclusion

SSMs are formidable neural network architectures capable of processing long-range sequences with excellent parameter efficiency. In this work, we analyze the compression of the memory footprint of the S4D [20] SSM towards hardware deployment with constrained resources at the edge. In the context of a common sequence classification task (sMNIST), we analyze the criticality of each parameter in the SSM network by applying PTQ to the SSM parameters at different bit-widths. We show that the precision of the state matrix $\bar{A}$ and state x are critical to performance. When using only PTQ, these values require at least 8-bit precision to perform well in small scale models, which remains true even with larger network layers. For this reason, we perform QAT to reduce the bit-width requirement for the SSM parameters. We demonstrate that it is indeed possible to lower the precision of $\bar{A}$ and x below 8 bits without catastrophic model failure, but 8-bit precision represents a reasonable compromise between memory footprint compression and performance. We develop a heterogeneous quantization scheme whereby different SSM parameters are quantized at different bit-precisions. To better optimize memory usage on hardware, our quantization scheme exploits the fact that different SSM parameters have different sensitivities to quantization. Combining QAT with the heterogeneous quantization technique, we recover a classification accuracy of 96% with QAT compared to 40% after PTQ, while reducing the full-precision model's memory footprint by a 6x factor. This scheme strikes a balance between high performance, low memory footprint and computational complexity, optimizing the S4D model for execution on an edge processor.

A. Outlook

This work paves the way towards the deployment of efficient yet powerful state-space models in edge computing cases. Different SSM variants have been developed, and it remains to be seen which is the most adapted to edge machine learning, presenting the best compromise between memory footprint, number of operations, and performance in relevant tasks. The S4D [4] model offers efficient execution thanks to the diagonal structure in the A matrix. The same holds for the S5 [24] and S7 [25] models, while the S6 alternative also simplifies the non-linearity to a binarized one, potentially helping reduce the number of operations at inference. The S4 [2] model and Mamba [26] are instead interesting options for natural language processing tasks.

The heterogeneous quantization scheme detailed in our approach incurs some computational overhead to align the integer ranges. Future work will address the hardware cost of on-device dequantization and quantization operations.

Furthermore, such quantized models remain to be tested on relevant tasks in the context of edge AI. An example is the Google Speech Command [27] benchmark tasks, a keyword spotting task. Other examples are bio-medical signal processing (ECG [28], EEG, and EMG [29]) or the processing of sensory signals from robots and drones.

Acknowledgment

We acknowledge the entire EIS lab for their support throughout the development of this project. The presented work has received funding from the Swiss National Science Foundation Starting Grant Project UNITE (TMSGI2-211461).

References

[1] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, "Hippo: Recurrent memory with optimal polynomial projections," *Advances in neural information processing systems*, vol. 33, pp. 1474–1487, 2020.

[2] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces. arxiv 2021," *arXiv preprint arXiv:2111.00396*.

[3] A. Gu, I. Johnson, K. Goel, K. Saab, T. Dao, A. Rudra, and C. Ré, "Combining recurrent, convolutional, and continuous-time models with linear state space layers," *Advances in neural information processing systems*, vol. 34, pp. 572–585, 2021.

[4] A. Gu, K. Goel, A. Gupta, and C. Ré, "On the parameterization and initialization of diagonal state space models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 35971–35983, 2022.

[5] M. Schöne, N. M. Sushma, J. Zhuge, C. Mayr, A. Subramoney, and D. Kappel, "Scalable event-by-event processing of neuromorphic sensory signals with deep state-space models," in *2024 International Conference on Neuromorphic Systems (ICONS)*, pp. 124–131, IEEE, 2024.

[6] K. Miyazaki, Y. Masuyama, and M. Murata, "Exploring the capability of mamba in speech applications," *arXiv preprint arXiv:2406.16808*, 2024.

[7] Y. Du, X. Liu, and Y. Chua, "Spiking structured state space model for monaural speech enhancement," in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 766–770, 2024.

[8] X.-T. Tran, L. Le, Q. T. Nguyen, T. Do, and C.-T. Lin, "Eeg-ssm: Leveraging state-space model for dementia detection," *arXiv preprint arXiv:2407.17801*, 2024.

[9] S. Li, T. Zhu, F. Duan, L. Chen, H. Ning, and Y. Wan, "Harmamba: Efficient wearable sensor human activity recognition based on bidirectional selective ssm," *arXiv e-prints*, pp. arXiv–2403, 2024.

[10] T. Suwannaphong, F. Jovan, I. Craddock, and R. McConville, "Optimising tinyml with quantization and distillation of transformer and mamba models for indoor localisation on edge devices," *Scientific Reports*, vol. 15, no. 1, p. 10081, 2025.

[11] M.-I. Stan and O. Rhodes, "Learning long sequences in spiking neural networks," *Scientific Reports*, vol. 14, no. 1, p. 21957, 2024.

[12] S. Karilanova, M. Fabre, E. Neftci, and A. Özçelikkale, "Zero-shot temporal resolution domain adaptation for spiking neural networks," *arXiv preprint arXiv:2411.04760*, 2024.

[13] S. Ghamari, K. Ozcan, T. Dinh, A. Melnikov, J. Carvajal, J. Ernst, and S. Chai, "Quantization-guided training for compact tinyml models," *arXiv preprint arXiv:2103.06231*, 2021.

[14] P.-E. Novac, G. Boukli Hacene, A. Pegatoquet, B. Miramond, and V. Gripon, "Quantization and deployment of deep neural networks on microcontrollers," *Sensors*, vol. 21, no. 9, p. 2984, 2021.

[15] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2704–2713, 2018.

[16] S. Abreu, J. E. Pedersen, K. M. Heckel, and A. Pierro, "Q-s5: Towards quantized state space models," *arXiv preprint arXiv:2406.09477*, 2024.

[17] H.-Y. Chiang, C.-C. Chang, N. Frumkin, K.-C. Wu, and D. Marculescu, "Quamba: A post-training quantization recipe for selective state space models," *arXiv preprint arXiv:2410.13229*, 2024.

[18] Y. Li, X. Liu, J. Li, R. Xu, Y. Chen, and Z. Xiong, "Qmamba: Post-training quantization for vision state space models," *arXiv preprint arXiv:2501.13624*, 2025.

[19] A. Pierro and S. Abreu, "Mamba-ptq: Outlier channels in recurrent large language models," *arXiv preprint arXiv:2407.12397*, 2024.

[20] S. M. Meyer, P. Weidel, P. Plank, L. Campos-Macias, S. B. Shrestha, P. Stratmann, and M. Richter, "A diagonal structured state space model on loihi 2 for efficient streaming sequence processing," *arXiv preprint arXiv:2409.15022*, 2024.

[21] G. Orchard, E. P. Frady, D. B. D. Rubin, S. Sanborn, S. B. Shrestha, F. T. Sommer, and M. Davies, "Efficient neuromorphic signal processing with loihi 2," in *2021 IEEE Workshop on Signal Processing Systems (SiPS)*, pp. 254–259, IEEE, 2021.

[22] A. Gu, I. Johnson, A. Timalsina, A. Rudra, and C. Ré, "How to train your hippo: State space models with generalized orthogonal basis projections," *arXiv preprint arXiv:2206.12037*, 2022.

[23] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[24] J. T. Smith, A. Warrington, and S. W. Linderman, "Simplified state space layers for sequence modeling," *arXiv preprint arXiv:2208.04933*, 2022.

[25] T. Soydan, N. Zubić, N. Messikommer, S. Mishra, and D. Scaramuzza, "S7: Selective and simplified state space layers for sequence modeling," *arXiv preprint arXiv:2410.03464*, 2024.

[26] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[27] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[28] E. Kim, J. Kim, J. Park, H. Ko, and Y. Kyung, "Tinyml-based classification in an ecg monitoring embedded system," *Computers, Materials and Continua*, vol. 75, no. 1, pp. 1751–1764, 2023.

[29] E. Donati, S. Benatti, E. Ceolini, and G. Indiveri, "Long-term stable electromyography classification using canonical correlation analysis," in *2023 11th International IEEE/EMBS Conference on Neural Engineering (NER)*, pp. 1–4, IEEE, 2023.